Modelos para el aprendizaje automático



Silvia N. Pérez

KNN: *k-nearest neighbors*

Bibliografía:

- (ISLR) An Introduction to Statistical Learning: with Applications in R (Springer Texts in Statistics)
- Nwanganga, F.; Chapple, M. Practical Machine Learning in R (2020).

K-NN: k vecinos más próximos



El algoritmo de los k vecinos más próximos (*k-nearest neighbors*) es un algoritmo que permite realizar **clasificación supervisada.**

Esto es, requiere conocer la "etiqueta" (valor declarado de la para cada caso) en el conjunto de entrenamiento.

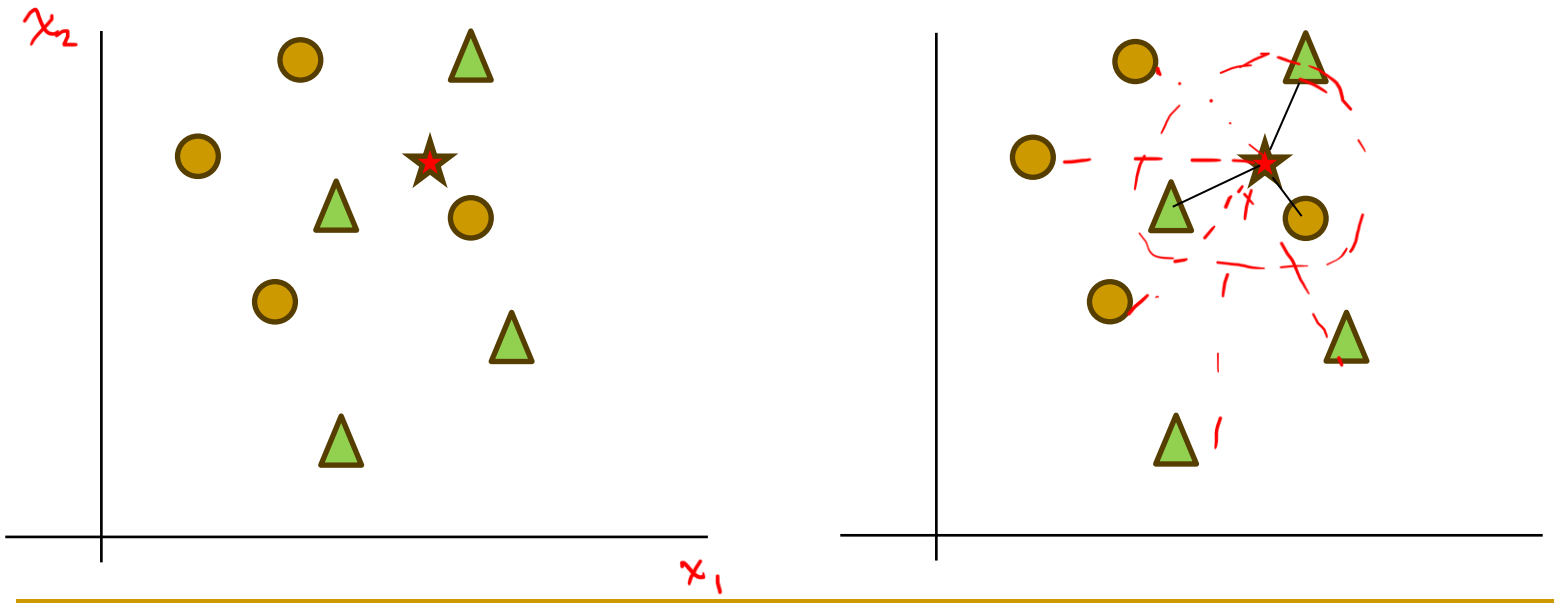
Es un método no paramétrico, no construye un modelo.

Es un método simple y efectivo, competitivo con algoritmos más complejos.

K-NN: cómo clasifica?

Veamos un ejemplo gráfico con 2 predictores. Tenemos un conjunto de casos ya clasificados (conjunto de entrenamiento) y queremos clasificar un caso nuevo.

Si tomamos **K=3** considero los 3 casos más cercanos.
Hay 2 ▲ y 1 ●, entonces clasifico como ▲



Observaciones

- Se requiere elegir **k** previamente! Se sugiere k impar para evitar empates.
- Hay que decidir la distancia a considerar. Lo usual es la euclídea pero permite Manhattan, Minkowsky…
- **Las variables deben ser todas numéricas,** aunque pueden usarse categóricas convertidas a numéricas ficticiamente (dummies).
- Se recomienda normalizar o estandarizar las variables numéricas previamente.

K-nn: pros

- No hace suposiciones de distribución de las variables.
- No “aprende” un modelo, es sólo un algoritmo.
- No le afecta multicolinealidad.

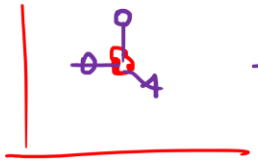
K-nn: contras

- Requiere datos completos.
- Es sensible al ruido: ver outliers!
- Con muchas predictoras, es muy costoso computacionalmente.
- K grande da resultados más estables, pero tiene mayor costo.

Como dosifica

Tomo como 'base' los casos de entrenamiento donde conozco "y" = dose objetivo

caso 1 de Test → 5



→ dos 0 y uno 1 ⇒ dosifica como 0

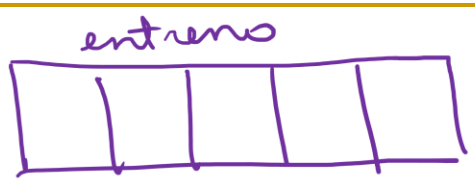
prob = $\frac{2}{3}$ = 0,66

caso 4to de Test → 23



→ tres 0 y ningún 1 ⇒ dosifica como 0

prob = $\frac{3}{3}$ = 1





hago CV con 5 partes

Para cada k de la grilla:

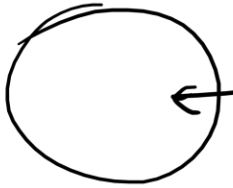


c/caso de) lo clasifica según los otros 4 partes → clasifico todos los casos de entreno



⇒ comparo c/lo real y testeo accuracy

k	accu	kappa
5		
7		
.		
.		
.		
1		
1		



← elijo k para q' sea máximo accu (mínimo error)

Vamos a R

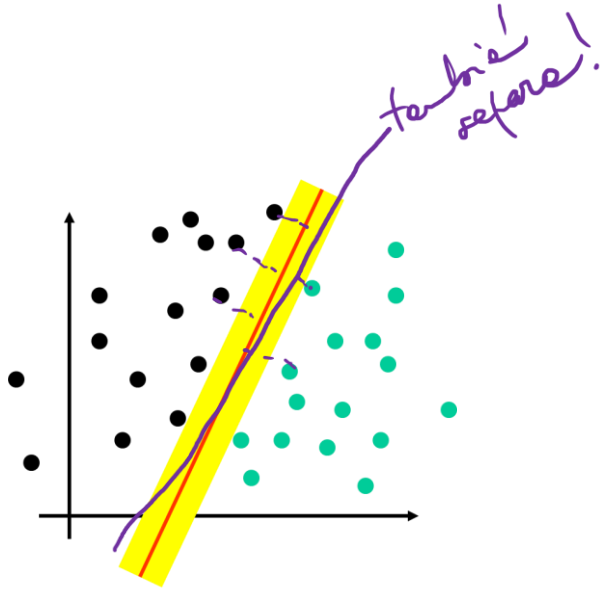
Introducción a Máquinas de vector soporte (SVM)

Bibliografía:

- (ISLR) An Introduction to Statistical Learning: with Applications in R (Springer Texts in Statistics)
- Support Vector Machines Succinctly by Alexandre Kowalczyk

Separando con hiperplanos

En principio, se buscaba un hiperplano que tenga separe.
Esto es, se mide la mínima distancia a las observaciones.



Separando con hiperplanos

Luego, se elige el de mayor margen de separación: el *maximal margin hyperplane.*

Las observaciones marcadas son las que definen el hiperplano. Se les llama *vectores soporte*.

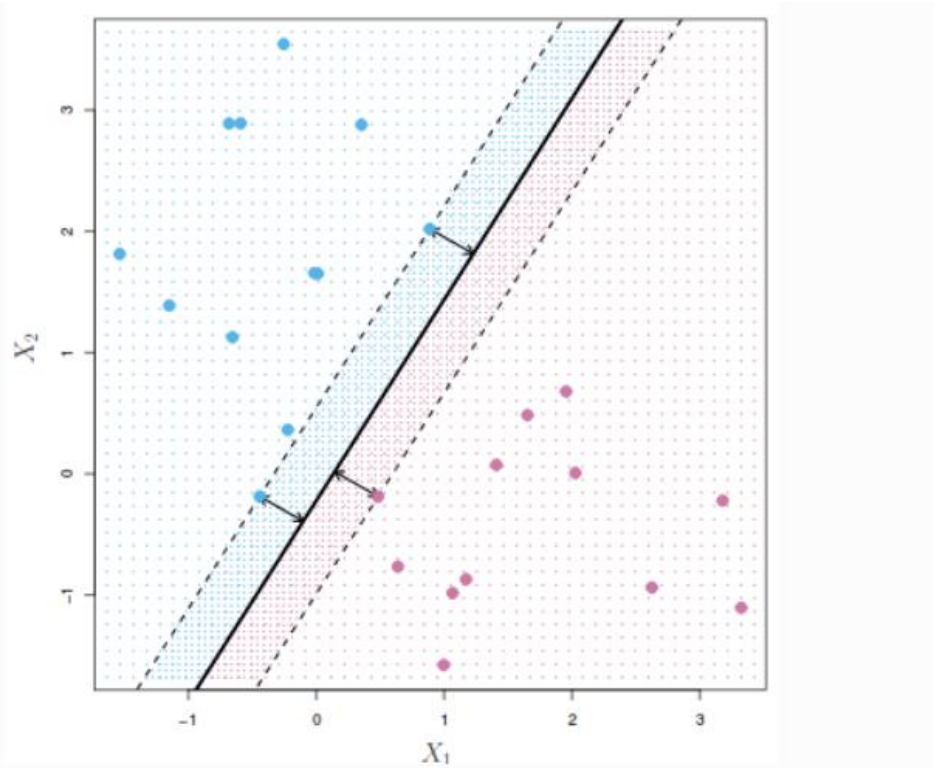


Imagen maximal margin hyperplane obtenida del libro ISLR

Hiperplano de margen maximal

Se presentan dos inconvenientes:

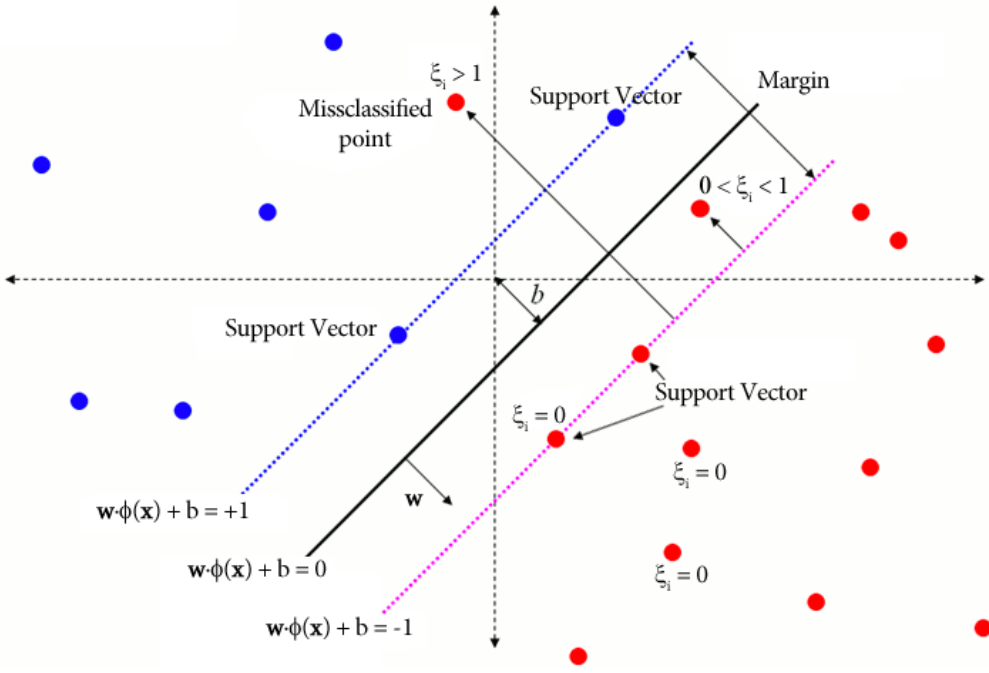
- El hiperplano tiene que separar perfectamente las observaciones, entonces es muy sensible a variaciones en los datos (poco robusto).
- Si el *maximal margin hyperplane* ajusta perfectamente a las observaciones de entrenamiento para separarlas todas correctamente, puede tener problemas de overfitting.

Se prefiere entonces obtener un hiperplano que 'casi' separe las clases, pero permitiendo que cometa unos pocos errores.

Esto corresponde al ***Support Vector Classifier***

Support Vector Classifier (Soft clasificador)

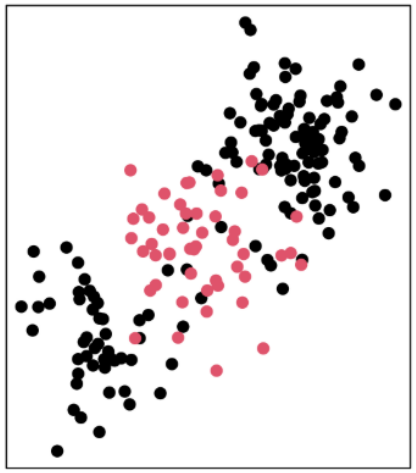
Se busca un hiperplano ajustando un hiperparámetro de *tuning* C.
C controla el número y severidad de las violaciones del margen que se toleran.
Si C=∞ , no se permite ninguna violación del margen y por lo tanto, el resultado es equivalente al Maximal Margin Classifier.
Si C se aproxima a cero, menos se penalizan los errores y más observaciones pueden estar en el lado incorrecto del margen o incluso del hiperplano.
(*C = cost en R y se encuentra por CV*)

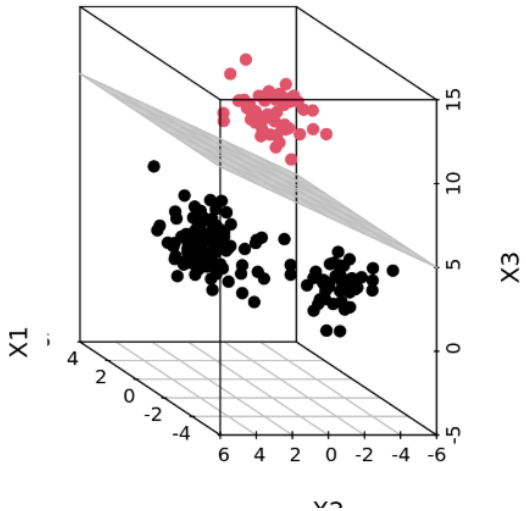


El hiperplano depende únicamente de una pequeña proporción de observaciones: los vectores soporte.
Esto da robustez frente a observaciones muy alejadas del hiperplano.

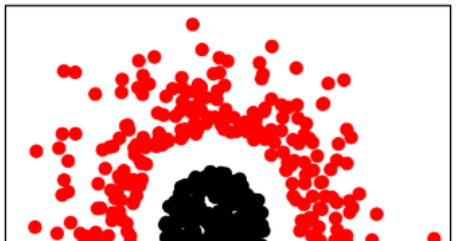
SVM: máquinas de vector soporte

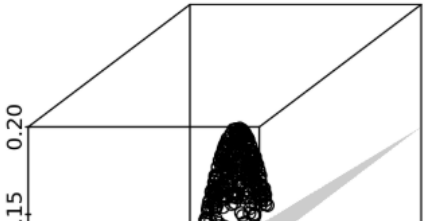
Lo anterior funciona bien si puede lograrse una (casi) separación mediante un hiperplano, o sea cuando el límite de separación entre clases es aproximadamente lineal.
La estrategia es entonces separarlos en una dimensión mayor. Esto se logra mediante *kernels.*





Tomado de Cienciadedatos.net





SVM kernels

Hiperparámetros

Lineal	$\langle x, x' \rangle = \sum_{i=1}^{n} x_i x'_i$	ninguno
Polinómico	$\gamma(1 + \langle x, x' \rangle)^d$	γ, d = grado
Radial	$\exp\left(\gamma \|x - x'\|^2\right)$	γ
Sigmoideo	$\tanh(k_1 \|x - x'\| + k_2)$	k_1, k_2

SVM para clasificación multiclase

One-versus-one: genera un total de K(K-1)/2 SVMs, comparando todos los posibles pares de clases.

Para generar una predicción se emplean cada uno de los clasificadores. Se clasifica que la observación pertenece a la clase a la que ha sido asignada con más frecuencia.

(svm( ) utiliza esta para clasificación multiclase)

One-versus-all: genera K SVMs distintos, cada uno comparando una de las K clases frente a las restantes K-1 clases. Se obtiene un hiperplano de clasificación para cada clase.

SVM: pros

- No hace suposiciones de distribución de las variables.
- Funciona bien con muchos atributos, aún mayor al número de casos.
- Eficiente en memoria porque usa sólo pocos puntos de entrenamiento en la decisión (puntos soporte).
- Robusto a atípicos.
- No se ve afectada por multicolineridad.

SVM: contras

- No da probabilidades.
- No funciona bien si hay mucho desbalance.
- Sensible a las escalas (se recomienda escalar los datos).
- No funciona bien si hay mucha “mezcla” de casos (difícil decidir el kernel!)

Vamos a R